MEMENTO SQL – PLSQL

Rappel : Les [ ] désignent les éléments facultatifs

Syntaxe générale d'un ordre SELECT:

```
SELECT [DISTINCT] col1 [[AS] alias_col1] [, col2 …. ]
  FROM tab1 [alias1] [, tab2 …]
  [WHERE conditions ]
  [GROUP BY col1 [, col2 ...] ]
  [HAVING condition_de_sélection_groupes]
  [ORDER BY col1 [DESC] [ ,col2 .. [DESC] ] ;
```

Opérateurs mathématiques : **=** , <, >, <=, >=, <> ou != (différent)

Opérateurs spécifiques : [NOT] LIKE, BETWEEN .. AND .., IN (…), IS [NOT] NULL

Opérateurs logiques : AND, OR, NOT

Caractères génériques : % (n car.) et _ (1 car) **Concaténation** : ||

Fonctions ORACLE	SYSDATE : date du jour (y compris l'heure, minutes et secondes) MONTHS_BETWEEN (date1, date2) : nb de mois entre les dates TO_DATE (ch, format) : transforme **ch** en date selon le format SUBSTR(ch, pos,lg) : extrait de **ch**, à partir de la position **pos**, **lg** car.
Fonctions d'agrégat	COUNT (colonne) COUNT (*) COUNT(DISTINCT colonne) SUM (colonne) AVG (colonne) MAX (colonne) MIN (colonne)
Insertions	INSERT INTO tab [(col1 , col2, ...)] VALUES (val1, val2...); INSERT INTO tab [(col1, col2, ...)] SELECT ... ;
Suppressions	DELETE FROM tab [WHERE ...];
Modifications	UPDATE tab SET col1 =val_1 [, col2=val_2 …] [WHERE condition];
Creation de table	CREATE TABLE tab (col1 type_col1 [NOT NULL |NULL] [DEFAULT val1] [contrainte_col1 [, ...]], col2 type_col2 [NOT NULL |NULL] [DEFAULT val1] [contrainte_col2 [, ...]], ... [, contrainte_tab [, ...]]) ; où **type_col** : NUMBER, VARCHAR2(n), INTEGER, DATE … où **contrainte_col** : [CONSTRAINT pk_col] PRIMARY KEY [CONSTRAINT fk_col] REFERENCES tab (col) [ON DELETE CASCADE|SET NULL] {CONSTRAINT ck_col] CHECK (condition_utilisant_col) où **contrainte_tab** : [CONSTRAINT pk_col] PRIMARY KEY(col1 [,col2 …]) [CONSTRAINT fk_col] FOREIGN KEY(col) REFERENCES tab(col) [ON DELETE CASCADE|SET NULL] [CONSTRAINT ck_col] CHECK (condition)
Modification de table	ALTER TABLE nom_table ADD col_nom type_col [NOT NULL |NULL] [DEFAULT val] ADD constraint_tab [, constraint_tab …] DROP CONSTRAINT constraint_nom MODIFY col_nom type_col |DEFAULT val | NULL | NOT NULL DROP COLUMN col_nom ;
Création de vue	CREATE [OR REPLACE] VIEW nom_vue AS SELECT ... [WITH CHECK OPTION];

Déclencheurs – Procédures – Fonctions

```
CREATE [OR REPLACE] TRIGGER nom_décl
{AFTER|BEFORE} événement [OR événement]
ON nom_table
[FOR EACH ROW]
[WHEN condition]
[DECLARE declarations]
 BEGIN
    Instructions;
 END;
```

Déclencheurs LMD

événement INSERT ou UPDATE [OF..] ou DELETE

nom_table table à laquelle le déclencheur est lié

condition : Si vraie, traitement exécuté (pas de req. SQL).

FOR EACH ROW et WHEN : uniquement pour décl. LIGNE

```
CREATE [OR REPLACE] PROCEDURE nom_proc
[(arg [mode] typologie [,arg [mode]
typologie ]…)]
[IS]
BEGIN
    Instructions;
END;
```

```
CREATE [OR REPLACE] FUNCTION nom_fct
[(arg [IN] typologie [,arg [mode] typologie
]…)]
RETURN typologie_retour [IS]
[Var typologie;]
BEGIN
    Instructions;
    RETURN Var;
END;
```

Généralités PL/SQL

Structure de contrôle conditionnelle :

```
IF conditions THEN
    Instructions ;
[ELSIF conditions THEN
    Instructions ;]
[ELSE
    Instructions ;]
END IF ;
```

Section EXCEPTION :

Principales erreurs ORACLE prédéfinies :

NO_DATA_FOUND – TOO_MANY_ROWS – DUP_VAL_ON_INDEX – OTHERS

Principales erreurs ORACLE non prédéfinies :

– 2290 : Violation de contrainte de vérification (CHECK)
– 2291 : Violation de contrainte d'intégrité : parent introuvable
– 2292 : Violation de contrainte d'intégrité : fils existant

Section Déclarations

Exemples de déclarations de variables de différents types avec possibilité d'initialisation

déclaration d'un caractère initialisé avec la lettre A

```
v_char      CHAR(1)   :=  'A';
```

déclaration d'un entier initialisé avec le nombre 1

```
v_int            INTEGER  :=  0;
```

déclaration d'une ch. de caractères

```
v_chaine    VARCHAR2(100) ;
```

Déclaration comme un champ d'une table

```
v_champ     TABLE.champ%TYPE ;
```

où champ est un champ de la table existante TABLE

Curseurs explicites avec déclaration :

```
DECLARE
CURSOR C IS (SELECT chp1, …
               FROM … WHERE …);
v_chp1 …%TYPE; -- var de réception
BEGIN
   OPEN C; --ouverture curseur
   LOOP
        FETCH C INTO v_chp1,….;
        EXIT WHEN C%NOTFOUND;
        Intructions;
   END LOOP;
   … utilisation C%ROWCOUNT…
   CLOSE C; -- Fermeture curseur
   …
END;
```

Curseurs explicites sans déclaration (FOR) :

```
 FOR Cur IN (SELECT chp1, … FROM … WHERE …)
   LOOP
        Instructions;
        … Cur.chp1 …
   END LOOP;
```

Instructions : Exemples d'Instructions pouvant être utilisées en PL/SQL

```
DBMS_OUTPUT.PUT_LINE (ch);              -- affiche la chaine ch
SELECT … INTO … FROM .. WHERE … ;       -- Récupère une ligne dans une table
a := a + 1 ;                            -- incrémentation de a
RAISE_APPLICATION_ERROR (numErr; msg)   -- déclenche une erreur (numErr entre -20000 et -20999)
```